STUDENT TECHNICAL REPORT / SUBMISSION RECORD

Assignment 3 - Real-Time Time Series Analysis and Anomaly Detection

Astana Weather and Urban Comfort Monitoring

This report records the implementation and evidence for the supplied synthetic Astana weather stream. The project focuses on PM2.5 smoothing, anomaly detection, method comparison, and a concise operational recommendation.

1. Student and Submission Information

Required field	Student entry
Student full name	Amankos Danial Dauletuly
Student ID	240260
Group / cohort	BDA-2405
Instructor	Serek Azamat
Submission date	19.10.2026
Individual variant / task code	240260
Git repository / project link	https://github.com/subdragon34/Time-Series-Anomaly-Detection.git
Python / tool versions	Python 3.10 compatible; pandas 2.3.3, NumPy 2.2.6, Matplotlib 3.10.6 (requirements.txt)

2. Work Record

Item	Student entry
Work start date	17 October 2026
Work completion date	19 October 2026
Main libraries / technologies	Python, Pandas, NumPy, Matplotlib
Source files / notebook	main.py
Test / experiment files	evidence/sensitivity_experiment.csv; evidence/calculation_check.json
Final repository commit/version	https://github.com/subdragon34/Time-Series-Anomaly-Detection.git

3. Report Structure

3.1 Time-Series Exploration

Time is converted to a Pandas datetime index before analysis. Temperature and wind change only slightly, while humidity generally declines. PM2.5 rises gradually from 28 to 47, then changes abruptly to 95 at 12:50, 110 at 12:55, and 118 at 13:00. Before applying any algorithm, I identify 12:50-13:00 as the likely anomaly interval. The series is too short to support a reliable seasonality conclusion.

Required evidence	Student response / reference
Code / file / function reference	main.py - load_data(), create_plots()
Result / experiment evidence	plots/01_temperature_timeseries.png to plots/04_humidity_timeseries.png; evidence/main_output.txt
Student-specific observation	The main abrupt change is isolated to PM2.5, while the other variables remain comparatively gradual.

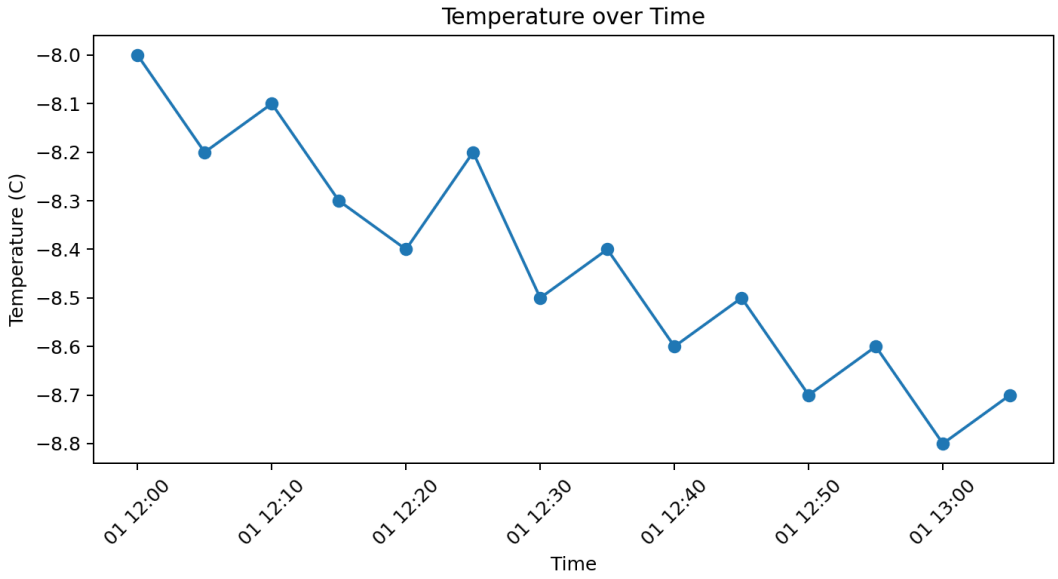


Figure 1. Temperature over time.

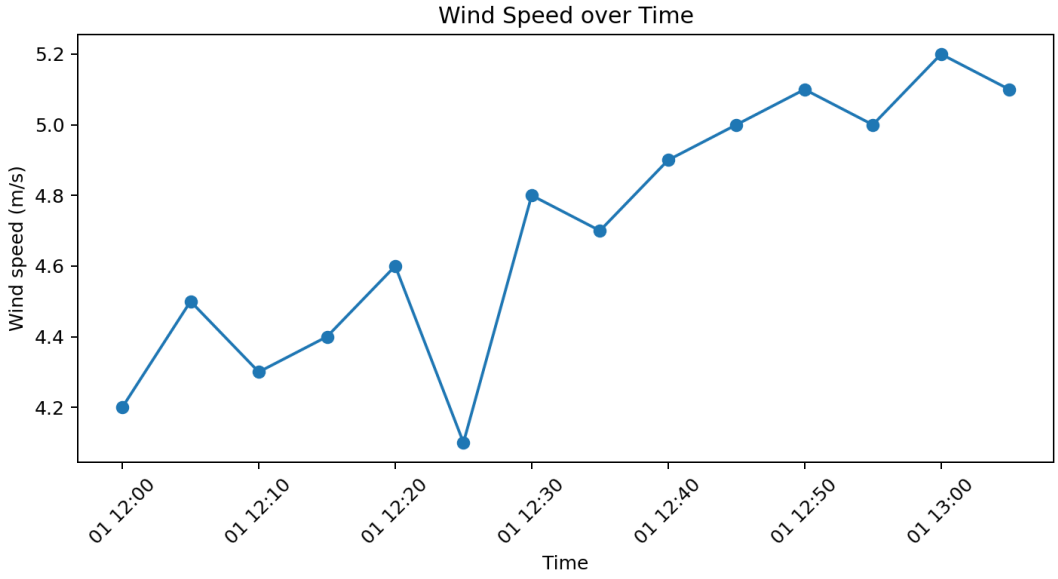


Figure 2. Wind speed over time.

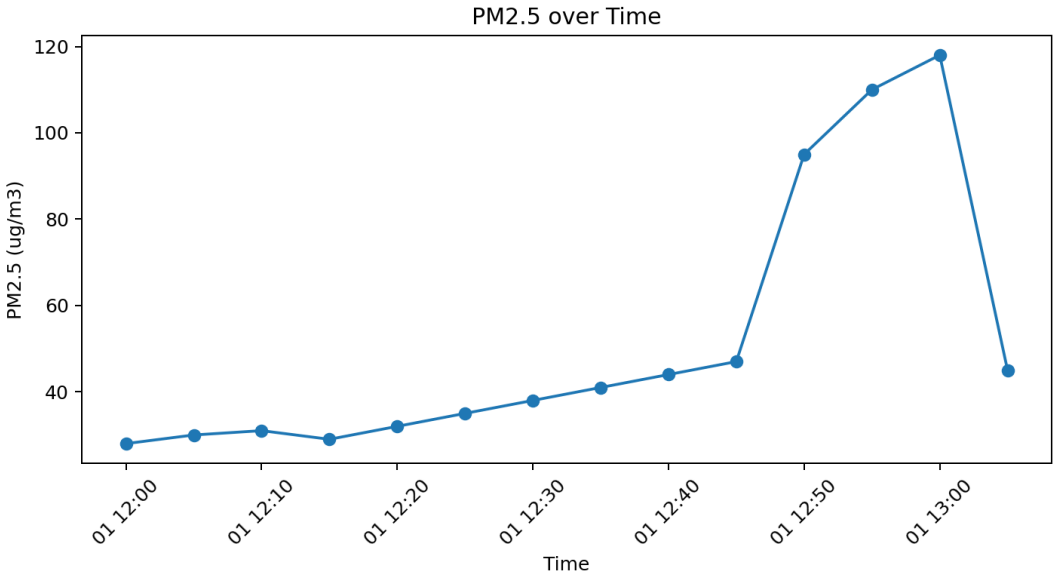


Figure 3. PM2.5 over time.

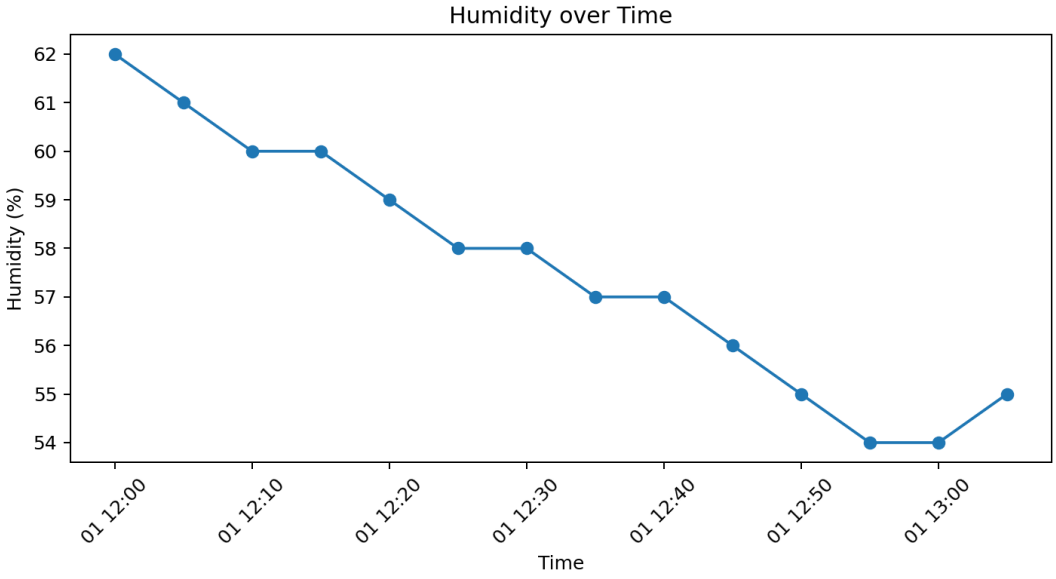


Figure 4. Humidity over time.

3.2 Moving Average and Exponential Smoothing

PM2.5 is smoothed in two ways. The moving average uses the current and previous two observations. Exponential smoothing uses alpha = 0.3, so the new observation receives 30% weight and the previous smoothed value receives 70%. Difference is reported as observed PM2.5 minus the smoothed value.

Time	PM2.5	3-point MA	Smoothed	Difference
12:10	31	29.667	29.320	1.680
12:15	29	30.000	29.224	-0.224
12:20	32	30.667	30.057	1.943
12:25	35	32.000	31.540	3.460
12:30	38	35.000	33.478	4.522
12:35	41	38.000	35.734	5.266
12:40	44	41.000	38.214	5.786
12:45	47	44.000	40.850	6.150
12:50	95	62.000	57.095	37.905
12:55	110	84.000	72.966	37.034
13:00	118	107.667	86.477	31.523

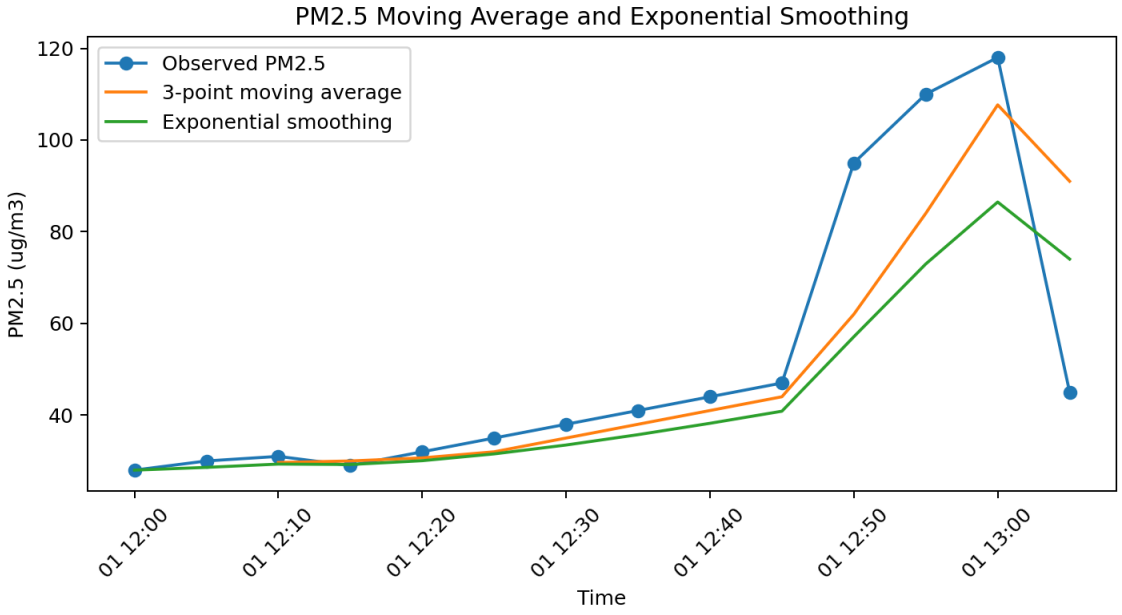


Figure 5. PM2.5 moving average and exponential smoothing.

Required evidence	Student response / reference
Code / file / function reference	main.py - add_smoothing()
Result / calculation evidence	processed_weather_data.csv; completed_tables.md; Figure 5
Student-specific observation	The sharp 12:50 increase produces a much larger gap between observed and smoothed PM2.5 than the earlier gradual changes.

3.3 Statistical Anomaly Detection

The rolling detector uses six successive PM2.5 readings sampled five minutes apart. Its current observation is included in the rolling mean and sample standard deviation (ddof=1). The required rule is abs(x - rolling_mean) > 2 x rolling_std. This is a simple local method, but the current spike also raises the threshold.

Time	Observed	Rolling mean	Rolling std	Threshold	Anomaly?
12:35	41	34.333	4.546	9.092	No
12:40	44	36.500	5.612	11.225	No
12:45	47	39.500	5.612	11.225	No
12:50	95	50.000	22.450	44.900	Yes
12:55	110	62.500	31.488	62.976	No
13:00	118	75.833	35.695	71.391	No

At 12:50, observed PM2.5 is 95, rolling mean is 50.000, rolling standard deviation is 22.450, and the 2-standard-deviation threshold is 44.900. The absolute difference is 45.000, so 45.000 > 44.900 and the point is flagged.

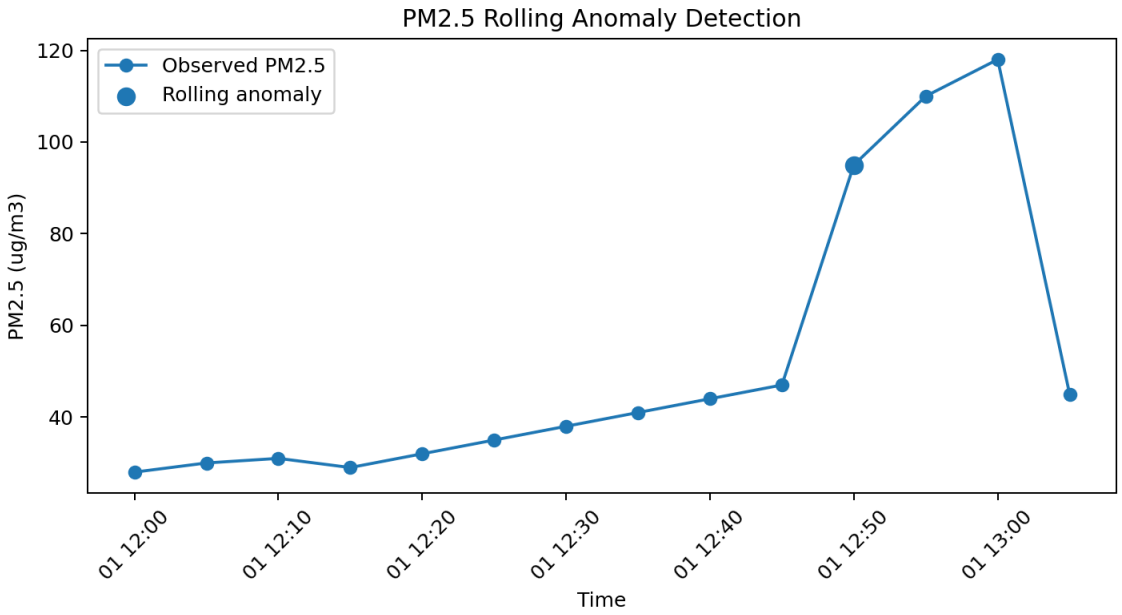


Figure 6. Rolling anomaly detection.

Required evidence	Student response / reference
Code / file / function reference	main.py - add_rolling_anomaly()
Result / calculation evidence	processed_weather_data.csv; evidence/calculation_check.json; Figure 6
Student-specific observation	The detector flags the first sharp jump at 12:50, then adapts upward as high values enter the rolling window.

3.4 Comparison of Detection Methods

The second method is a global z-score with an absolute cutoff of 2. The rolling rule flags 12:50, while the global z-score flags 13:00. The rolling rule reacts earlier to this local change. The global z-score uses the full 14-reading dataset, so it is an offline comparison, not a directly deployable live detector. There are no labeled normal/anomalous observations, so true false-positive counts cannot be verified.

Method	Detected points	False positives observed	Advantages	Limitations
Rolling threshold	12:50	Not verifiable (no labels)	Local and fast; threshold can be shown directly	Adapts after the spike and misses later high points
Global z-score	13:00	Not verifiable (no labels)	Simple standardized score	Whole-series statistics are not directly live

Interpretability: both methods have simple formulas; the rolling threshold provides a recent baseline suitable for display. Reaction speed: the rolling rule flags the first jump at 12:50, versus 13:00 for the global z-score. Live suitability: the rolling method can update as readings arrive, while a global z-score would require a reference distribution available before the new point.

Required evidence	Student response / reference
Code / file / function reference	main.py - add_zscore_method(), sensitivity_experiment()
Result / experiment evidence	evidence/sensitivity_experiment.csv; evidence/main_output.txt
Student-specific observation	Reducing the rolling multiplier from 2.0 to 1.5 flags several earlier gradual increases, so I keep 2.0 for a more selective alert.

3.5 Operational Recommendation

The smart-campus dashboard should display current PM2.5, the recent PM2.5 line, rolling mean, rolling threshold, and anomaly status. The statistical alert should trigger when abs(PM2.5 - rolling_mean) is greater than 2 x rolling_std. The alert should appear as a visible dashboard warning and be sent to the responsible campus operations staff.

Required evidence	Student response / reference
Code / file / function reference	main.py - operational recommendation output in main()
Result / experiment evidence	evidence/main_output.txt
Student-specific observation	I use the assignment's statistical rule as the alert condition instead of introducing an unsupported external PM2.5 health threshold.

3.6 Technical Reflection

My main methodological choice is a six-reading window with the specified 2 x standard-deviation threshold. In the sensitivity experiment, reducing the multiplier from 2.0 to 1.5 flags more of the gradual rise. The dataset has only 14 synthetic points, and the spike increases the rolling standard deviation, causing later high readings to be missed. For a longer stream I would test a past-only baseline and measure false alerts using labeled events.

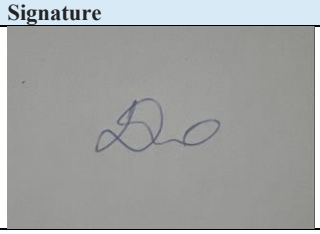
Required evidence	Student response / reference
Code / file / function reference	main.py - ROLLING_WINDOW, ROLLING_MULTIPLIER, sensitivity_experiment()
Result / experiment evidence	evidence/sensitivity_experiment.csv
Student-specific observation	A 1.5 multiplier is more sensitive but less selective, so 2.0 is retained.

4. Individual Work and Anti-Plagiarism Record

Individual evidence	Student entry
My unique technical/design decision	Use six consecutive PM2.5 readings for the required rolling rule; compare multipliers 2.0 and 1.5 for sensitivity.
My own test/experiment	Sensitivity check: 2.0 flags 12:50; 1.5 flags 12:25, 12:30, 12:50 and 12:55.
Problem/bug I personally encountered and solved	Unexpected result investigated: after 12:50, the rolling standard deviation jumps, so later high readings are not flagged. Checked intermediate mean/std values and documented this limitation; the required rule remains unchanged.
Exact code/repository/commit reference	main.py: load_data(), add_smoothing(), add_rolling_anomaly(), add_zscore_method(), sensitivity_experiment(); add Git URL/commit before submission.
One limitation of my implementation	Fourteen synthetic readings, and an inclusive rolling window can mask consecutive spikes; no labels for verified false-positive counts.
One improvement I propose	Use a past-only baseline and validate its alert quality over longer, labeled live data.

5. Student Declaration of Originality

I confirm that this submission represents my own work. I have not copied another student's source code, calculations, tables, architecture, plots, screenshots, logs, results, or written explanations. Any external material or reused code has been identified. I understand that the instructor may request a live demonstration, code modification, reproduction of calculations, or explanation of any submitted component.

Role	Name	Signature	Date
Student / Executor	Amankos Danial Dauletuly		19.10.2026

Appendix - Included Evidence

evidence/main_output.txt - complete execution output

evidence/calculation_check.json - intermediate numeric values and detector parameters

evidence/sensitivity_experiment.csv - 2.0 versus 1.5 multiplier experiment

evidence/part_b_table.csv; evidence/part_c_table.csv - exported table calculations

plots/01_temperature_timeseries.png to plots/06_anomaly_detection.png - generated figures